1. Bias-Variance Tradeoff

One of the most important concepts in Machine Learning.

Bias

Bias is the error caused by overly simple assumptions in the model.

Example

Using a straight line to fit a curved dataset.

High Bias = Underfitting

- Model is too simple
- Cannot learn patterns
- Poor training accuracy
- Poor testing accuracy

Example:

```
from sklearn.linear_model import LinearRegression
```

A simple linear model may not capture complex relationships.

Variance

Variance is the error caused by sensitivity to training data.

High Variance = Overfitting

- Model memorizes training data
- Excellent training accuracy
- Poor testing accuracy

Example:

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=None)
```

A very deep tree may memorize the data.

Bias-Variance Tradeoff

Goal:

```
Low Bias + Low Variance
```

Situation	Bias	Variance
Underfitting	High	Low
Good Model	Medium	Medium
Overfitting	Low	High

Example

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score

X,y = make_regression(n_samples=1000,n_features=5,noise=20)

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42
)

model = DecisionTreeRegressor()

model.fit(X_train,y_train)

print("Train Score:",model.score(X_train,y_train))
print("Test Score:",model.score(X_test,y_test))
```

Output

```
Train Score: 1.00
Test Score: 0.72
```

Meaning:

```
Training = Perfect
Testing = Poor

=> Overfitting
=> High Variance
```

---

2. Feature Engineering

Creating useful features from existing data.

Original Data:

Age
20

Age
30
40

New Feature:

Age **Age²**
20 400
30 900
40 1600

Code

```
import pandas as pd

df = pd.DataFrame({
    "Age":[20,30,40]
})

df["Age_Square"] = df["Age"]**2

print(df)
```

Output

```
   Age  Age_Square
0   20         400
1   30         900
2   40        1600
```

---

3. Encoding Categorical Variables

Machine Learning models understand numbers, not text.

Dataset:

Gender
Male
Female
Male

Label Encoding

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
```

```
df = pd.DataFrame({
    "Gender":["Male","Female","Male"]
})

le = LabelEncoder()

df["Gender_Encoded"] = le.fit_transform(df["Gender"])

print(df)
```

Output

```
   Gender  Gender_Encoded
0    Male              1
1  Female              0
2    Male              1
```

One-Hot Encoding

```
import pandas as pd

df = pd.DataFrame({
    "Gender":["Male","Female","Male"]
})

encoded = pd.get_dummies(df)

print(encoded)
```

Output

```
   Gender_Female  Gender_Male
0              0            1
1              1            0
2              0            1
```

---

4. Feature Scaling

Bringing all features to same scale.

Before Scaling:

Age	Salary
20	20000
40	100000

Salary dominates Age.

Standardization

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Code

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

data = [[20,20000],
        [30,50000],
        [40,100000]]

scaler = StandardScaler()

scaled = scaler.fit_transform(data)

print(scaled)
```

Output

```
[[-1.22 -1.07]
 [ 0.00 -0.27]
 [ 1.22  1.34]]
```

Mean becomes 0 and Std becomes 1.

---

5. Normalization

Converts values between 0 and 1.

Formula:

$$x' - \frac{x - min}{max - min}$$

Code

```
from sklearn.preprocessing import MinMaxScaler

data = [[20],[30],[40]]

scaler = MinMaxScaler()

scaled = scaler.fit_transform(data)
```

```
print(scaled)
```

Output

```
[[0.0]
 [0.5]
 [1.0]]
```

---

6. Feature Selection Techniques

Selecting important features.

Benefits:

- Faster training
- Better accuracy
- Less overfitting

Example

```
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2

iris = load_iris()

X = iris.data
y = iris.target

selector = SelectKBest(score_func=chi2,k=2)

X_new = selector.fit_transform(X,y)

print(X.shape)
print(X_new.shape)
```

Output

```
(150,4)
(150,2)
```

Selected only 2 best features.

---

7. Ensemble Learning

Combining multiple models.

Instead of:

```
1 Model
```

Use:

```
Many Models
     ↓
Combined Prediction
```

Advantages:

- Higher accuracy
- Better generalization

---

8. Random Forest

Random Forest = Many Decision Trees

Example:

```
Tree1 → Cat
Tree2 → Cat
Tree3 → Dog
Tree4 → Cat

Final = Cat
```

Majority voting.

Classification Example

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

iris = load_iris()

X = iris.data
y = iris.target

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42
)

model = RandomForestClassifier(
    n_estimators=100,
```

```
    random_state=42
)

model.fit(X_train,y_train)

print("Accuracy:",model.score(X_test,y_test))
```

Output

```
Accuracy: 1.0
```

---

9. Boosting Concepts

Boosting trains models sequentially.

Previous mistakes are corrected by next model.

Popular Algorithms:

1. AdaBoost
2. Gradient Boosting
3. XGBoost
4. LightGBM
5. CatBoost

AdaBoost Example

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import load_iris

iris = load_iris()

X = iris.data
y = iris.target

model = AdaBoostClassifier(
    n_estimators=50,
    random_state=42
)

model.fit(X,y)

print(model.score(X,y))
```

Output

```
0.98
```

---

10. Hyperparameter Tuning

Hyperparameters are set before training.

Examples:

```
n_estimators=100
max_depth=5
k=3
learning_rate=0.1
```

---

11. Grid Search

Checks all possible combinations.

Example:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()

params = {
    "n_estimators":[10,50,100],
    "max_depth":[2,4,6]
}

grid = GridSearchCV(
    RandomForestClassifier(),
    params,
    cv=5
)

grid.fit(iris.data,iris.target)

print(grid.best_params_)
```

Output

```
{
'n_estimators':100,
'max_depth':4
}
```

Total combinations:

```
3 × 3 = 9
```

All tested.

---

12. Random Search

Randomly tests combinations.

Faster than Grid Search.

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

iris = load_iris()

params = {
    "n_estimators":[10,50,100,200],
    "max_depth":[2,4,6,8]
}

search = RandomizedSearchCV(
    RandomForestClassifier(),
    param_distributions=params,
    n_iter=5
)

search.fit(iris.data,iris.target)

print(search.best_params_)
```

Output

```
{
'n_estimators':200,
'max_depth':6
}
```

Only 5 random combinations tested.

---

13. Cross Validation

Instead of one train-test split:

```
Train = 80%
Test = 20%
```

Use multiple splits.

Example:

```
Fold1
Fold2
Fold3
```

```
Fold4
Fold5
```

Average accuracy is calculated.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()

scores = cross_val_score(
    RandomForestClassifier(),
    iris.data,
    iris.target,
    cv=5
)

print(scores)
print("Average:",scores.mean())
```

Output

```
[1.00 0.96 0.93 1.00 0.96]

Average:
0.97
```

Meaning:

```
97% average accuracy
```

---

14. Model Pipelines

Pipeline automates:

```
Scaling
 ↓
Feature Selection
 ↓
Model Training
```

in one step.

Without Pipeline

```
scaler.fit_transform(X)

selector.fit_transform(X)

model.fit(X,y)
```

Many steps.

With Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

iris = load_iris()

pipeline = Pipeline([
    ("scaler",StandardScaler()),
    ("selector",SelectKBest(k=2)),
    ("model",RandomForestClassifier())
])

pipeline.fit(
    iris.data,
    iris.target
)

print(
    pipeline.score(
        iris.data,
        iris.target
    )
)
```

Output

```
0.98
```

Complete Interview Definitions

Topic	Definition
Bias	Error due to simple model
Variance	Error due to sensitivity to training data
Feature Engineering	Creating useful features from data
Encoding	Converting text into numbers
Scaling	Standardizing feature values
Normalization	Converting values between 0 and 1

Topic	Definition
Feature Selection	Selecting important features
Ensemble Learning	Combining multiple models
Random Forest	Collection of Decision Trees
Boosting	Sequential learning from mistakes
Hyperparameter Tuning	Finding best model settings
Grid Search	Tests all parameter combinations
Random Search	Tests random parameter combinations
Cross Validation	Multiple train-test evaluations
Pipeline	Automates ML workflow